

Swinburne University of Technology

School of Science, Computing and Engineering Technologies

ASSIGNMENT COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: 2, Indexers, Method Overriding, and Lambdas
Due date: **13th May 2026, 23:59**
Lecturer: Ms Siti Hawa

Your name: Khant Thu Aung_____ **Your student id:** 105292912_____

Section	C1	C2	C3
	★		

Marker's comments:

Problem	Marks	Obtained
1	48	
2	30+10= 40	
3	58	
Total	146	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener:_____

SOLUTION:

IntVector.Cpp

Problem 1

```
#include "IntVector.h"
#include <stdexcept>

// Constructor: copy argument array
IntVector::IntVector(const int aArrayOfIntegers[], size_t aNumberOfElements)
{
    fNumberOfElements = aNumberOfElements;

    fElements = new int[fNumberOfElements];

    for (size_t i = 0; i < fNumberOfElements; i++)
    {
        fElements[i] = aArrayOfIntegers[i];
    }
}

// Destructor: release memory
IntVector::~~IntVector()
{
    delete[] fElements;
}

// size getter
size_t IntVector::size() const
{
    return fNumberOfElements;
}

// element getter
const int IntVector::get(size_t aIndex) const
{
    return (*this)[aIndex];
}

// swap two elements within the vector
void IntVector::swap(size_t aSourceIndex, size_t aTargetIndex)
{
    if (aSourceIndex >= fNumberOfElements || aTargetIndex >= fNumberOfElements)
    {
        throw std::out_of_range("Illegal vector indices");
    }
}
```

```

    int temp = fElements[aSourceIndex];
    fElements[aSourceIndex] = fElements[aTargetIndex];
    fElements[aTargetIndex] = temp;
}
// indexer
const int IntVector::operator[](size_t aIndex) const
{
    if (aIndex >= fNumberOfElements)
    {
        throw std::out_of_range("Illegal vector index");
    }

    return fElements[aIndex];
}

```

```

~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P1 main 76 > g++ IntVector.cpp Main.cpp -o p1
~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P1 main 76 > ./p1
Test range check:
Properly caught error: Illegal vector index
Test swap:
lVector[3] = 86
lVector[6] = 20
lVector.get( 3 ) = 20
lVector.get( 6 ) = 86
Properly caught error: Illegal vector indices

```

Problem 2

SortableIntVector.cpp

```
#include "SortableIntVector.h"

// Constructor
SortableIntVector::SortableIntVector(const int aArrayOfIntegers[], size_t aNumberOfElements):
IntVector(aArrayOfIntegers, aNumberOfElements){}

// Bubble Sort
void SortableIntVector::sort(Comparable aOrderFunction)
{
    for (size_t i = 0; i < size(); i++)
    {
        for (size_t j = 0; j < size() - i - 1; j++)
        {
            if (!aOrderFunction(get(j), get(j + 1)))
            {
                swap(j, j + 1);
            }
        }
    }
}
```

Main_PS2.cpp (Cut only Problem 2 Part)

```
#ifdef P2

#include "SortableIntVector.h"

void runP2()
{
    int IArray[] = { 34, 65, 890, 86, 16, 218, 20, 49, 2, 29 };
    size_t IArrayLength = sizeof(IArray) / sizeof(int);

    SortableIntVector IVector( IArray, IArrayLength );

    cout << "Bubble Sort:" << endl;

    cout << "Before sorting:" << endl;

    for ( size_t i = 0; i < IVector.size(); i++ )
    {
```

```

        cout << lVector[i] << ' ';
    }

    cout << endl;

    // Use a lambda expression here that orders integers in increasing order.
    // The lambda expression does not capture any variables or throws any exceptions.
    // It has to return a bool value.
    lVector.sort([](int aLeft, int aRight) { return aLeft <= aRight; });
// (10)

    cout << "After sorting:" << endl;

    for ( size_t i = 0; i < lVector.size(); i++ )
    {
        cout << lVector[i] << ' ';
    }
    cout << endl;
}

#endif

```

```

~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P2 main ?6 > g++ IntVector.cpp Main.cpp SortableIntVector.cpp -o p2
~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P2 main ?6 >
~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P2 main ?6 > ./p2
Bubble Sort:
Before sorting:
34 65 890 86 16 218 20 49 2 29
After sorting:
2 16 20 29 34 49 65 86 218 890

```

Problem 3

ShakerSortableIntVector.cpp

```
#include "ShakerSortableIntVector.h"

// Constructor
ShakerSortableIntVector::ShakerSortableIntVector(const int aArrayOfIntegers[],size_t aNumberOfElements):
SortableIntVector(aArrayOfIntegers, aNumberOfElements){}

// Cocktail Shaker Sort
void ShakerSortableIntVector::sort(Comparable aOrderFunction)
{
    if (size() <= 1)
    {
        return;
    }

    size_t left = 0;
    size_t right = size() - 1;

    while (left < right)
    {
        // Move from left to right
        for (size_t i = left; i < right; i++)
        {
            if (aOrderFunction(get(i), get(i + 1)))
            {
                swap(i, i + 1);
            }
        }

        right--;

        // Move from right to left
        for (size_t i = right; i > left; i--)
        {
            if (aOrderFunction(get(i - 1), get(i)))
            {
                swap(i - 1, i);
            }
        }

        left++;
    }
}
```

```
~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P3 main ?9 > g++ Main.cpp ShakerSortableIntVector.cpp SortableIntVector.cpp IntVector.cpp -o p3
~/Desktop/ProblemSets/COS30008Assignments/ProblemSet2/P3 main ?8 > ./p3
Cocktail Shaker Sort:
Before sorting:
34 65 890 86 16 218 20 49 2 29
After sorting:
890 218 86 65 49 34 29 20 16 2
```